

Conversational Memory & Context Feature

Overview

The Property RAG System now includes **Conversational Memory & Context**, enabling multi-turn conversations where the system remembers previous queries and understands contextual follow-up questions.

Key Features

1. Session Management

- Each conversation gets a unique session ID
- Sessions persist across queries
- Automatic session timeout (60 minutes of inactivity)
- Create new conversations or clear existing ones

2. Context Awareness

- System remembers previous queries in a conversation
- Understands follow-up questions like:
 - "What about cheaper ones?"
 - "Show me those in London instead"
 - "How about 2 bedrooms?"
- Automatically applies context from previous interactions

3. Smart Query Enhancement

- Detects follow-up questions using indicators
- Enhances queries with context from conversation history

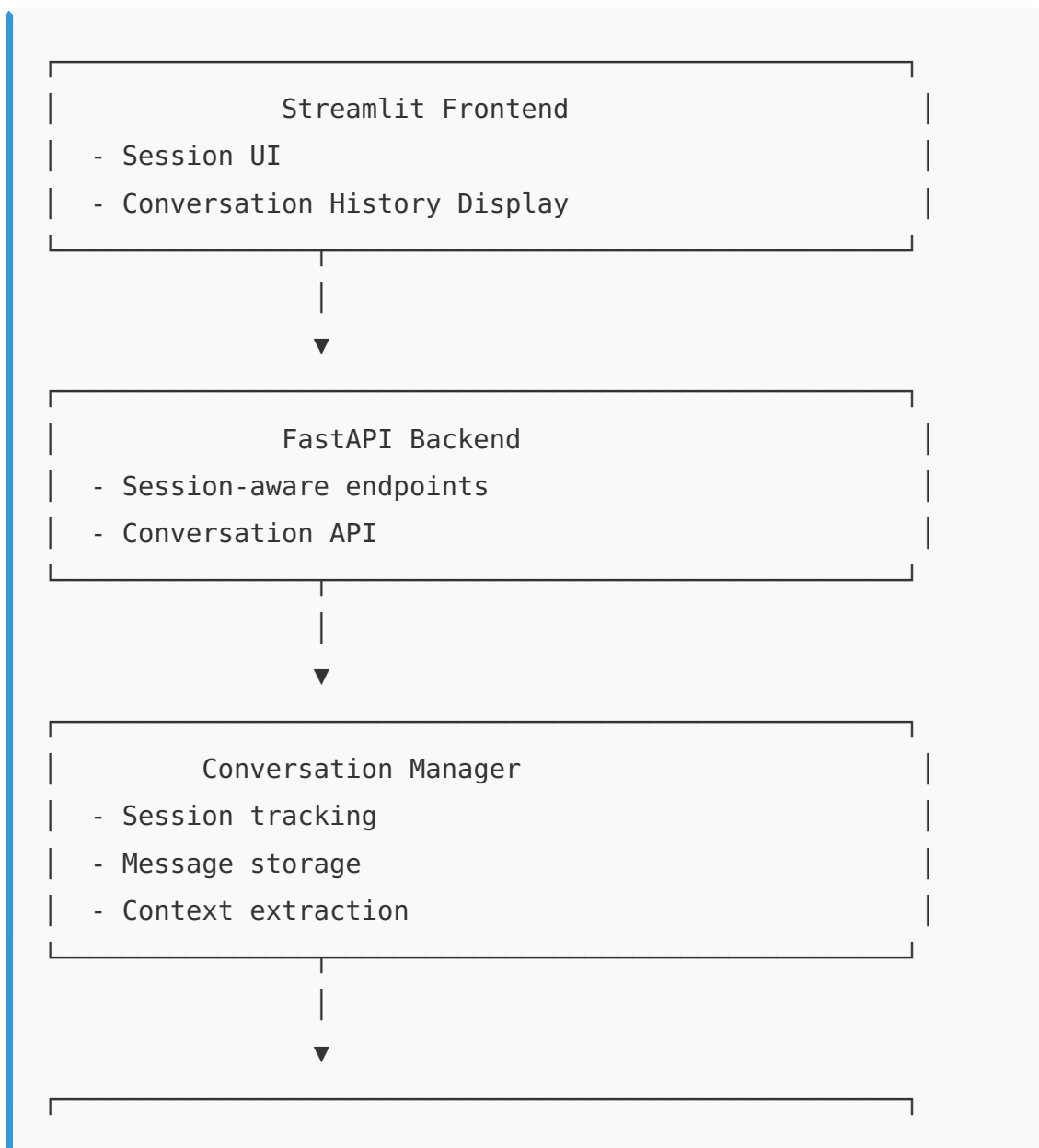
- Merges filters from previous queries when appropriate

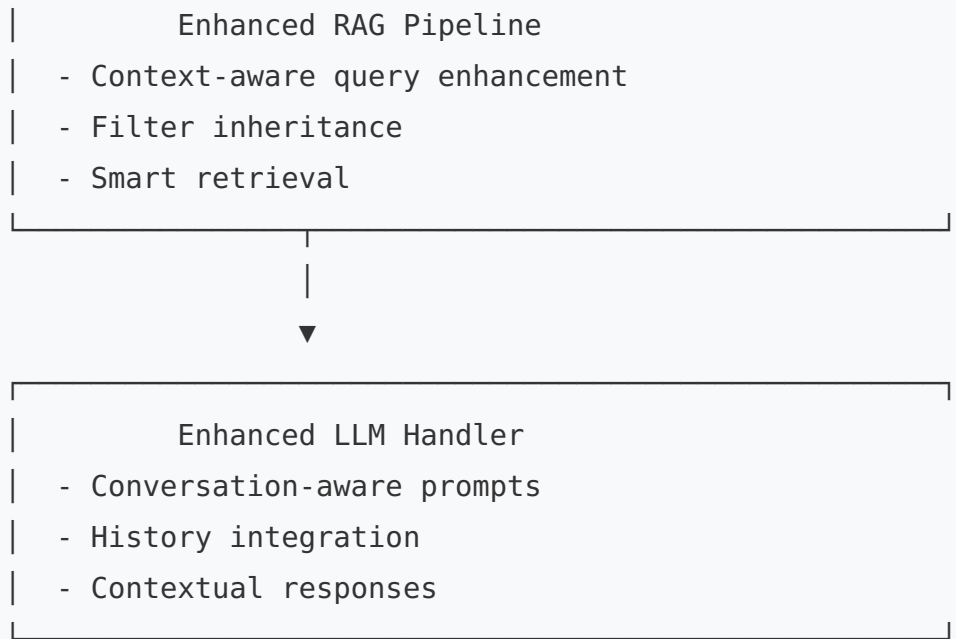
4. Conversation History

- Stores full conversation history
- View past messages in the chat
- Persistent storage to disk
- Analytics on conversation patterns

Architecture

Components





New Files

backend/conversation_manager.py

Main conversation management system with: - `ConversationMessage` : Represents a single message - `ConversationSession` : Manages a single conversation - `ConversationManager` : Global manager for all sessions

Key Classes

```
# Message structure
class ConversationMessage:
    role: str # 'user' or 'assistant'
    content: str
    metadata: dict
    timestamp: datetime

# Session management
class ConversationSession:
    session_id: str
    messages: List[ConversationMessage]
```

```
metadata: dict # query_count, filters_used, topics, etc.

def add_message(...)
def get_conversation_history(...)
def get_context_summary(...)
def extract_context_from_previous(...)
```

API Endpoints

New Conversation Endpoints

1. POST /query (Enhanced)

Now accepts `session_id` parameter:

```
{
  "query": "What about cheaper ones?",
  "n_results": 5,
  "session_id": "uuid-here"
}
```

Response includes:

```
{
  "answer": "...",
  "properties": [...],
  "session_id": "uuid-here",
  "has_conversation_context": true
}
```

2. POST /conversation/new

Create a new conversation session:

```
curl -X POST http://localhost:8000/conversation/new
```

Response:

```
{
  "session_id": "abc-123-def",
  "message": "New conversation session created"
}
```

3. GET /conversation/{session_id}/history

Get conversation history:

```
curl http://localhost:8000/conversation/{session_id}/history?limit=10
```

Response:

```
{
  "session_id": "abc-123-def",
  "messages": [
    {
      "role": "user",
      "content": "Show me 2 bedroom apartments"
    },
    {
      "role": "assistant",
      "content": "Found 5 properties..."
    }
  ],
  "count": 2
}
```

4. DELETE /conversation/{session_id}

Clear a conversation:

```
curl -X DELETE http://localhost:8000/conversation/{session_id}
```

5. GET /conversation/stats

Get statistics about all conversations:

```
curl http://localhost:8000/conversation/stats
```

Response:

```
{
  "active_sessions": 5,
  "total_messages": 127,
  "total_queries": 64,
  "avg_messages_per_session": 25.4
}
```

6. GET /conversation/{session_id}/context

Get contextual information for a session:

```
curl http://localhost:8000/conversation/{session_id}/context
```

Frontend Features

New UI Elements

1. **Conversation Controls**
2.  New Chat - Start fresh conversation
3.  Clear - Clear current conversation
4. Session status indicator
5. **Conversation History**
6. Expandable history viewer
7. Shows all previous messages
8. Timestamps and metadata

9. Context Indicators

10. Visual feedback when using context

11. "💡 Using conversation context" message

Session State Management

```
st.session_state['session_id'] # Current session ID
st.session_state['conversation_history'] # Local history cache
st.session_state['query_count'] # Message counter
```

How It Works

Example Conversation Flow

User Query 1:

```
"Show me 2 bedroom apartments under £1500"
```

System: - Creates new session (if none exists) - Searches for 2BR apartments under £1500 - Stores query + response in session - Returns results with session_id

User Query 2:

```
"What about ones in Manchester?"
```

System: - Retrieves session context - Detects follow-up question (mentions "what about") - Extracts previous filters: bedrooms=2, max_price=1500 - Adds location filter: Manchester - Enhanced query: "apartments Manchester 2 bedroom" - Searches with combined filters - Returns contextual response

User Query 3:

"Show me cheaper ones"

System: - Recognizes comparative request - Maintains previous filters and location - Adjusts price range downward - Provides refined results

Context Extraction Logic

The system automatically extracts:

1. **Previous Filters**
2. Price ranges
3. Bedroom/bathroom counts
4. Property types
5. **Mentioned Locations**
6. Cities, areas, neighborhoods
7. Extracted from query text
8. **Property Types**
9. Apartments, houses, studios, etc.
10. Used for query enhancement
11. **User Preferences**
12. Frequently used filters
13. Common query patterns

Follow-up Detection

System recognizes follow-up questions through indicators: - "what about", "how about", "show me" - "those", "these", "same", "similar" - "cheaper", "expensive", "bigger", "smaller" - "also", "instead", "but", "other", "more"



Data Storage

Session Persistence

Sessions are stored in: `backend/analytics/conversations/`

Each session saved as: `{session_id}.json`

```
{
  "session_id": "abc-123-def",
  "messages": [
    {
      "role": "user",
      "content": "...",
      "metadata": {...},
      "timestamp": "2025-10-08T10:30:00"
    }
  ],
  "created_at": "2025-10-08T10:25:00",
  "last_accessed": "2025-10-08T10:35:00",
  "metadata": {
    "query_count": 3,
    "total_properties_viewed": 15,
    "filters_used": ["bedrooms", "max_price"],
    "topics": ["apartments", "manchester"]
  }
}
```

Session Cleanup

- **Timeout:** 60 minutes of inactivity
- **Automatic cleanup:** Removes expired sessions
- **Max sessions:** 1000 active sessions (configurable)

Usage Examples

Basic Flow

1. **Start conversation:** `python # First query creates session automatically POST /query { "query": "Find 3 bedroom houses" } # Returns: session_id`
2. **Continue conversation:** `python POST /query { "query": "What about ones under £2000?", "session_id": "previous-session-id" } # Uses context from first query`
3. **View history:** `python GET /conversation/{session_id}/history`

Advanced Usage

```
import requests

# Create session
response = requests.post("http://localhost:8000/conversation/new")
session_id = response.json()["session_id"]

# Query 1
query1 = {
    "query": "Show me apartments in London",
    "session_id": session_id,
    "n_results": 5
}
result1 = requests.post("http://localhost:8000/query", json=query1)

# Query 2 (follow-up)
query2 = {
    "query": "What about 2 bedroom ones?",
    "session_id": session_id
}
result2 = requests.post("http://localhost:8000/query", json=query2)
# Automatically applies London filter from Query 1
```

```
# Get full conversation
history = requests.get(f"http://localhost:8000/conversation/{session_id}/history")
print(history.json())
```

Configuration

Conversation Manager Settings

```
# In backend/conversation_manager.py
ConversationManager(
    persist_directory="analytics/conversations", # Storage location
    session_timeout_minutes=60, # Session expiry time
    max_sessions=1000 # Maximum active sessions
)
```

Session Settings

```
# Per-session settings
ConversationSession(
    session_id="...",
    max_history=10 # Keep last 10 message pairs
)
```

Testing

Test Script

```
# Test conversation memory
from backend.conversation_manager import ConversationManager

manager = ConversationManager()
```

```
# Create session
session_id = manager.create_session()

# Add messages
manager.add_message_to_session(
    session_id,
    'user',
    'Show me 2 bedroom apartments',
    {'filters': {'bedrooms': 2}}
)

manager.add_message_to_session(
    session_id,
    'assistant',
    'Found 5 properties...'
)

# Get context
context = manager.get_session_context(session_id)
print(context)

# Get history
history = manager.get_conversation_history(session_id)
print(history)
```



Benefits

1. **Natural Conversations**
2. Users can ask follow-up questions naturally
3. No need to repeat context every time
4. **Better User Experience**
5. Feels like chatting with a person
6. Faster queries (less typing)

7. **Improved Results**

- 8. Context helps refine searches
- 9. Better understanding of user intent

10. **Analytics**

- 11. Track conversation patterns
- 12. Understand user behavior across sessions



Future Enhancements

Potential improvements:

1. **Long-term Memory**

- 2. Remember user preferences across sessions
- 3. Personalization based on history

4. **Multi-user Support**

- 5. User authentication
- 6. Private conversation histories

7. **Advanced Context**

- 8. Entity recognition
- 9. Sentiment analysis
- 10. Intent classification

11. **Conversation Analytics**

- 12. Common conversation flows
- 13. Drop-off analysis
- 14. Topic clustering

15. **Export Conversations**

- 16. Download chat history

17. Share conversations



Related Documentation

- [Main README](#)
- [API Documentation](#)
- [Setup Guide](#)



Troubleshooting

Session Not Persisting

- Check `backend/analytics/conversations/` directory exists
- Verify write permissions

Context Not Working

- Ensure `session_id` is passed in requests
- Check session hasn't expired (60 min timeout)
- Verify backend logs for context extraction

High Memory Usage

- Reduce `max_sessions` setting
- Lower `session_timeout_minutes`
- Reduce `max_history` per session



Support

For issues or questions about conversational memory: - Check backend logs: Look for "ConversationManager" messages - Verify session existence: `GET /conversation/{session_id}/context` - Test conversation flow: Use provided test scripts

Version: 2.0.0

Last Updated: October 8, 2025

Feature Status:  Production Ready